

■ Phase 2: Requirement Analysis

Solution Requirements

Date:	July 3, 2026
Team ID:	SWTID-2026-4823
Project Name:	BOOK STORE (BookNest)

Functional Requirements

Following are the functional requirements of the proposed online bookstore solution, mapped directly to our MERN stack controllers and database schemas.

FR No.	Requirement Epic	Description & Implementation Details
FR-1	User Management & Authentication	Support customer, seller, and admin signups and logins. Hash passwords using bcrypt and validate routes using authorization header JWT tokens. Default user status is 'approved'.
FR-2	Book Catalog Search & Filtering	Customers can retrieve books globally and filter by text search (title, author, tags) or select genres. Pricing filters and sort routines (price low-high, rating, newest) are handled on the server.
FR-3	Wishlist & Cart	Authenticated customers can add/remove books in their wishlist and cart. Item quantities can be adjusted, and total price is calculated client-side.
FR-4	Checkout & Order Tracking	Simulated checkout saves shipping details, computes shipping fees, creates an order document in the DB, and displays order tracking history under /orders/me.
FR-5	Seller Inventory Management	Sellers can add books, update prices/stocks, list items with cover images (uploaded via Multer), and view orders linked to their books in `/api/orders/inbox`.
FR-6	Admin Portal	Admins can access a unified dashboard (/api/admin/dashboard) displaying system summaries, delete listings, and approve/suspend seller accounts.

Non-functional Requirements

Following are the non-functional requirements targeting safety, speed, and platform scalability.

NFR No.	Non-Functional Requirement	Target & Implementation Plan
NFR-1	Usability	Responsive UI compatible with desktop, tablet, and mobile browsers, utilizing unified CSS stylesheets and intuitive navigation menus.
NFR-2	Security	Secure credentials hashing using bcrypt, validate token claims on backend API routes via jwt.verify, configure CORS origins strictly.
NFR-3	Reliability	Consistent backend error handling returning user-friendly messages (400/401/403/404/500). Database data constraints enforced via Mongoose schema validators.
NFR-4	Performance	Frontend initial load under 1.5 seconds via Vite code-splitting and asset bundling. Backend queries optimized by index mappings on MongoDB.

NFR-5	Availability	Continuous availability achieved by deploying stateless Express API on Render, static frontend assets on global CDNs, and database on MongoDB Atlas replica sets.
NFR-6	Scalability	3-tier separation (Client SPA, REST API, Atlas Cloud Database) ensures layers can scale independently under concurrent peak load.